

MycoFlow: A Bio-Inspired, Reflexive QoS System for Embedded OpenWrt Routers

Baris Can Atakli

Department of Software Engineering
Muğla Sıtkı Koçman University
Muğla, Turkey
bariscanatakli@posta.mu.edu.tr

Dr. Öğr. Üyesi Cihat Çetinkaya

Department of Software Engineering
Muğla Sıtkı Koçman University
Muğla, Turkey

Abstract—Consumer home networks must simultaneously serve latency-sensitive workloads (gaming, VoIP, video conferencing) and high-throughput bulk transfers on commodity routers with tight CPU, RAM, and flash constraints. CAKE AQM eliminates bufferbloat but cannot automatically classify encrypted traffic or adapt its bandwidth cap to real-time congestion. We present *MycoFlow*, a bio-inspired embedded QoS daemon for OpenWrt routers that runs a 1 Hz reflexive Sense→Infer→Act→Stabilize loop entirely on-device, classifying traffic at two complementary levels: a six-class per-device behavioral persona engine and a twelve-class per-flow service taxonomy driven by a three-signal weighted voter (passive DNS snooping, destination-port hints, behavioral fingerprints). Classified flows are steered into CAKE diffserv priority tins via a CONNMARK-based DSCP pipeline; a passive BPF TCP RTT observer feeds a per-flow auto-corrector, and a feedback-driven hysteresis controller adapts the WAN bandwidth cap. Four controlled experiments demonstrate: (i) a $154\times$ lower in-tin delay for gaming vs. bulk traffic under per-device DSCP marking; (ii) a 38% reduction in gaming latency under five simultaneous mixed-persona clients vs. CAKE-only; (iii) correct per-flow service separation when a single IP concurrently runs game, VoIP, bulk, and torrent traffic; and (iv) an in-vivo DNS ablation ($N = 1,650$ stratified-random flows, Universidad del Cauca dataset) showing passive transit-DNS lifts CAKE tin-matched accuracy from 30% to 70.4% ($p < 10^{-63}$, McNemar), with zero flows degraded by DNS evidence. The daemon runs at constant 1,152 KB RSS (static CONNMARK libs included) with median 6% CPU on a single Cortex-A53 core and writes zero bytes to NAND flash.

Index Terms—bufferbloat, CAKE, DSCP, eBPF, OpenWrt, persona-aware QoS, reflexive control, bio-inspired networking, home network, adaptive queue management, flow-aware classification, passive TCP RTT, conntrack, CONNMARK, service taxonomy

I. INTRODUCTION

The modern household router is the single most critical network device for millions of users, yet it typically operates a fixed, administrator-configured Quality of Service (QoS) policy that was set at install time and never updated. Meanwhile, the application mix on a home network changes continuously: a gaming session demands sub-5 ms latency, a video conference requires stable 200–400 kbps with jitter below 15 ms, a streaming client needs 8–25 Mbps with no packet loss, and a background BitTorrent process competes for any remaining bandwidth with hundreds of simultaneous TCP connections.

Bufferbloat [1]—the phenomenon where excess queuing inside network devices inflates round-trip time by hundreds of milliseconds—remains the dominant cause of degraded user experience on home networks. A single saturating TCP flow can fill a 1000-packet FIFO queue in under 100 ms on a 20 Mbit/s WAN link, rendering gaming and VoIP completely unusable until the flow terminates.

Active Queue Management (AQM) algorithms such as CoDel [4], fq_CoDel [3], and CAKE [2] have made significant progress against bufferbloat. CAKE in particular, which is the default AQM on OpenWrt since version 22.03, combines shaping, per-flow fair queuing, and Differentiated Services (DiffServ) priority tins in a single qdisc. However, two fundamental challenges remain unsolved for home deployments:

- **No automatic traffic classification.** CAKE’s diffserv mode relies on DSCP markings in IP headers. On home networks, most traffic arrives with CS0 (Best Effort) markings, so every flow—gaming, streaming, torrent—lands in the same tin regardless of its latency sensitivity.
- **Static WAN rate limit.** The CAKE bandwidth parameter is set once. It cannot react to congestion detected through probe-based measurements or queue depth signals.

We present **MycoFlow**, a lightweight C11 daemon that addresses both problems. Its design is inspired by the information-processing strategy of fungal mycelial networks [11]: a mycelium continuously senses its environment through hyphal tip probes, redistributes nutrients adaptively using hysteresis-controlled transport decisions, and restructures itself in response to persistent stress. MycoFlow translates these principles to the network control plane: periodic probe-based sensing, per-device persona inference from flow statistics, DSCP-based traffic steering into CAKE tins, and bandwidth adaptation governed by a hysteresis controller with sliding baseline tracking.

Contributions of this paper:

- 1) A six-class per-device persona engine and a twelve-class per-flow service taxonomy, both operating without DPI or machine learning, connected by a three-signal weighted voter (passive DNS snooping, destination-port hints, behavioral fingerprints) and a CONNMARK-based DSCP pipeline that steers flows into the correct CAKE priority tin for their full lifetime. In a $N=15$ multi-run

single-IP experiment, per-flow DSCP correctly fills the CAKE Voice tin with $14,193 \pm 21$ game/VoIP packets per run vs. 3 ± 1 under per-device assignment ($p < 10^{-40}$).

- 2) A passive BPF TCP RTT observer (TC clsact hook) and a feedback-driven hysteresis controller with sliding EWMA baseline for WAN bandwidth adaptation; the per-flow RTT demote/re-promote ladder is implemented and deployed but its quantitative impact is left for future evaluation.
- 3) A practical architectural fix for in-router DNS sniffing: `AF_INET SOCK_RAW` silently misses transit DNS from clients that bypass the local resolver; replacing it with `AF_PACKET SOCK_DGRAM ETH_P_IP` yields a +12.5-pt CAKE tin accuracy lift ($p < 10^{-44}$).
- 4) An in-vivo DNS signal ablation ($N = 1,650$ stratified-random flows, Universidad del Cauca dataset) with Wilson 95% CIs and paired McNemar testing, showing passive transit-DNS lifts CAKE tin-matched accuracy from 30% to **70.4%** with zero flows degraded and no temporal overfitting ($\Delta_{\text{cal} \rightarrow \text{holdout}} < 0.5$ pt).

The remainder of this paper is organized as follows. Section II provides background on bufferbloat and AQM. Section III surveys related work. Section IV presents the design rationale. Section V presents the system architecture. Section VI formalizes the control loop. Section VII describes the implementation. Section VIII reports evaluation results. Section IX discusses implications and limitations. Section X concludes.

II. BACKGROUND

A. Bufferbloat and AQM

When a WAN uplink is saturated, the router’s transmit queue accumulates packets faster than they can be sent. Without queue management, packets wait in FIFO order; round-trip time grows linearly with queue depth. On a 20 Mbit/s uplink with a 1000-packet queue and 1500-byte packets, the maximum queue drain time exceeds 600 ms—far beyond the 20 ms threshold for interactive applications.

CoDel [4] controls standing queue depth by measuring the minimum queuing delay experienced by packets over a sliding window; it signals congestion via ECN or drops when the minimum delay exceeds a target (typically 5 ms) for longer than an interval (100 ms). `fq_CoDel` [3] adds per-flow fair queuing so that a single bulk flow cannot monopolize the queue at the expense of interactive flows. CAKE [2] subsumes both and adds three layers of priority: per-tin bandwidth allocation, per-flow isolation within each tin, and per-host round-robin within each flow set.

B. CAKE diffserv4

CAKE’s `diffserv4` mode partitions traffic into four tins based on the IP Precedence field (upper three bits of DSCP): Voice (Precedence 4–7), Video (Precedence 2–3), Best Effort (Precedence 0), and Bulk (Precedence 1). Each tin has an independent minimum guaranteed rate and an AQM target delay. Under CAKE `diffserv4`, a packet with DSCP CS4 (Precedence

4) enters the Voice tin and is served before any Best Effort or Bulk packet. MycoFlow exploits this mechanism: by correctly marking per-device flows, it shifts traffic from the default Best Effort tin into the appropriate priority tier without any modification to CAKE itself.

The DSCP-to-tin mapping used by MycoFlow is: EF $\rightarrow$ Voice (VoIP), CS4 $\rightarrow$ Voice (gaming), CS3 $\rightarrow$ Video (video call), CS2 $\rightarrow$ Video (streaming), CS1 $\rightarrow$ Bulk (torrent, bulk transfer).

C. DiffServ and DSCP on Home Networks

The Differentiated Services architecture [6] defines per-hop behaviors based on the DSCP field of the IP header. Edge routers are expected to re-mark traffic entering the network, but most ISP CPE devices either ignore DSCP or clear it. Home devices typically generate unmarked traffic (CS0), and no mechanism exists to automatically reclassify it at the home gateway. MycoFlow fills this gap by inferring intent from behavioral signals rather than port numbers or payload inspection.

III. RELATED WORK

AQM evolution. Proportional Integral controller Enhanced (PIE) [5] achieves CoDel-equivalent performance with lower computational cost. FQ-PIE combines PIE with per-flow queuing. CAKE [2] remains the most comprehensive open-source AQM for home gateways, integrating shaping, fair queuing, and DiffServ. None of these systems perform automatic per-device DSCP classification.

Traffic classification. GGFASST [7] applies payload-agnostic deep learning to encrypted traffic fingerprinting at line rate; it achieves high accuracy but requires offline model training infrastructure and cloud-side catalog updates, making continuous adaptation incompatible with embedded routers. Lotfollahi et al. [8] use deep learning on raw packet features; again, resource requirements preclude embedded deployment. nDPI [9] provides open-source DPI classification but requires payload parsing, raising privacy concerns and significant CPU cost on constrained hardware. MycoFlow deliberately avoids both ML and DPI: its heuristic decision tree runs in under 1 μ s per device per cycle on a Cortex-A53 core.

Adaptive QoS and SQM. SDN-based per-application QoS systems achieve fine-grained traffic differentiation but require a network controller external to the CPE, adding deployment complexity incompatible with consumer home settings. OpenWrt `qosify` provides static DSCP mapping via nftables rules with fixed port-to-class tables, but no runtime sensing or adaptation.

SQM (Smart Queue Management, `luci-app-sqm`) is the canonical OpenWrt QoS solution [2]: it configures CAKE or `fq_CoDel` with user-specified bandwidth limits and applies AQM. However, SQM has three structural limitations that MycoFlow addresses. *First*, SQM requires the operator to manually configure the WAN bandwidth cap once at installation; it does not adjust the cap in response to observed congestion or time-of-day variation. *Second*, SQM provides no

TABLE I
COMPARISON OF RELATED QoS APPROACHES. EMB.=RUNS FULLY ON CPE; AUTO=AUTOMATIC CLASSIFICATION; NO ML=NO ML MODEL REQUIRED; NO DPI=NO PAYLOAD INSPECTION; ADAPT=RUNTIME ADAPTATION. [†]NDPI HAS AN OPENWRT PACKAGE BUT IMPOSES SIGNIFICANT CPU COST ON CONSTRAINED HARDWARE.

System	Emb.	Auto	No ML	No DPI	Adapt
SQM/CAKE	✓	×	✓	✓	×
qosify	✓	×	✓	✓	×
nDPI	✓ [†]	✓	✓	×	×
GGFAST	×	✓	×	✓	×
SDN-QoS	×	✓	✓	✓	✓
MycoFlow	✓	✓	✓	✓	✓

automatic device or flow classification: if a user wants gaming traffic to enter CAKE’s Voice tin, they must manually assign DSCP marks on each game client—an operation requiring per-device network configuration knowledge that is unavailable to most residential users. *Third*, SQM cannot distinguish between multiple applications running simultaneously on the same host (e.g., a gaming session and a background torrent), because it operates at the qdisc level with no per-flow state. MycoFlow is designed as a *zero-touch complement* to SQM: it deploys CAKE identically but adds autonomous sensing, classification, and actuation so that no manual DSCP tagging or bandwidth tuning is ever required. CAKE’s diffserv tier assignment is purely static once configured; no published mechanism exists to auto-learn per-flow DSCP marks at the kernel qdisc level without an external classifier.

eBPF for edge telemetry. Magnani et al. [10] demonstrate adaptive eBPF-based monitoring pipelines on edge routers, showing that kernel-side counters can expose per-flow statistics with negligible overhead. MycoFlow uses eBPF for passive TCP RTT sampling via a TC clsact hook but primarily relies on `/proc/net/nf_conntrack` for flow statistics, as eBPF kernel headers are not available on all OpenWrt builds.

Bio-inspired networking. Ant Colony Optimization [15] has been applied to routing in multi-hop networks but requires agent communication infrastructure absent in home routers. Adamatzky and colleagues [12] characterize mycelial networks as fault-tolerant distributed logic gates. Fricker et al. [11] model nutrient transport in mycelial networks as an adaptive flow optimization problem. Fukasawa et al. [13] analyze how foraging heuristics in mycelium maximize resource extraction under uncertainty. MycoFlow is the first system to apply mycelial foraging and transport principles specifically to the QoS control plane of an embedded router.

Positioning. MycoFlow occupies a unique design point: it runs *entirely on the CPE*, requires *no cloud or SDN controller*, uses *no ML or DPI*, and *automatically classifies and steers* per-device traffic. Table I summarizes the comparison.

IV. DESIGN RATIONALE

MycoFlow’s architecture is motivated by an analogy with fungal mycelial networks [11], [13], which face a structurally similar resource allocation problem: a single nutrient

TABLE II
MYCELIAL PRINCIPLES MAPPED TO MYCOFLOW

Biology	MycoFlow Mechanism
Hyphal tip sensing	ICMP multi-ping; Netlink qdisc stats; conntrack; passive DNS; BPF passive TCP RTT
Adaptive nutrient redistribution	Per-device and per-flow DSCP marking into CAKE priority tins (via iptables and CONNMARK/nftables)
Hysteresis & structural memory	Majority-vote window; two-tick flow stability gate; action cooldown timers
Load-triggered remodeling	Sliding EWMA baseline; step adaptation via action feedback ring; per-flow RTT demote/re-promote ladder

source must be distributed among competing consumers with heterogeneous requirements, under dynamic and partially observable conditions. We do not claim that our mechanisms are algorithmically *derived* from biology; rather, the analogy clarifies *why* particular engineering choices (local sensing, hysteresis, adaptive thresholds) are well-suited to the embedded QoS domain.

Mycelium achieves adaptive resource distribution through four mechanisms that motivate MycoFlow’s design (Table II):

The key property borrowed from mycelial biology is *local decision-making*: each routing decision in a mycelium depends only on local chemical gradients, not on a global network map. MycoFlow similarly requires no external state: all classification and control decisions derive from metrics collected on the router itself.

The mycelial analogy also motivates the choice of hysteresis as the primary stability mechanism. Mycelial networks do not oscillate between resource allocation states because transport channels have inertia: they require a sustained signal above a threshold before remodeling occurs. MycoFlow’s *k*-of-*m* majority vote (default $k = 3$, $m = 5$ cycles) and cooldown timers serve the same purpose.

V. SYSTEM ARCHITECTURE

A. Overview

MycoFlow consists of seven components deployed on the OpenWrt router: `mycoflowd` (the control daemon), a CAKE qdisc on the WAN interface, an optional CAKE qdisc on an IFB device for ingress shaping, an iptables mangle chain for per-device DSCP marking, an iptables mangle POSTROUTING chain that restores per-flow CONNMARKs to DSCP (via the `iptables-nft` frontend), an optional BPF TC clsact program that samples passive TCP RTT on the WAN interface, and the LuCI dashboard.

Fig. 1 shows the reflexive loop. The daemon runs at a configurable rate (default 1 Hz) and executes four phases per cycle:

- 1) **Sense:** Collect RTT/jitter via ICMP probes, interface byte/packet counters via `/proc/net/dev`, qdisc statistics via `NETLINK_ROUTE`, flow statistics from

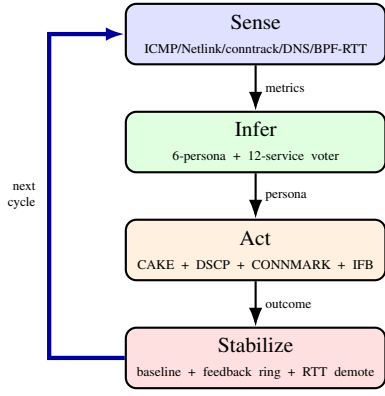


Fig. 1. MycoFlow reflexive control loop.

/proc/net/nf_conntrack, passive DNS responses from the local resolver, and per-flow smoothed TCP RTT from the BPF map.

- 2) **Infer:** Apply EWMA smoothing to raw metrics; run the six-class per-device persona engine and update the system-wide persona via majority vote; run the three-signal weighted voter over each active 5-tuple to assign a flow-level service class.
- 3) **Act:** If the persona changed, update CAKE AQM target latency and apply per-device DSCP mangle rules. For each newly stable flow, push a CONNMARK so the iptables mangle POSTROUTING chain restores its DSCP on egress. If a flow's RTT exceeds its service target for two ticks, demote it one tier; re-promote symmetrically after recovery. If congestion is detected, adjust the WAN bandwidth cap subject to cooldown constraints.
- 4) **Stabilize:** Record actuation outcomes in the feedback ring; evaluate ring for step-size adaptation; update the sliding baseline.

B. Module Structure

The daemon comprises 20 C11 source modules (Table III) totalling approximately 4500 lines of code, compiled for aarch64-openwrt-linux-musl via the OpenWrt SDK. The modules are grouped into three layers: (i) the core 1 Hz reflexive loop, (ii) the per-device persona layer, and (iii) the v3 flow-aware layer (classifier, mark engine, passive RTT observer) that classifies individual conntrack 5-tuples into service classes.

VI. THE CONTROL LOOP

A. Metric Sensing and Smoothing

RTT and jitter. Every cycle, `myco_sense` issues three ICMP echo requests to a configurable probe host (default 1.1.1.1), measuring mean RTT and standard deviation (jitter) from the replies. Packet loss is computed as $(N_{\text{sent}} - N_{\text{received}})/N_{\text{sent}}$. Raw measurements are smoothed by an Exponentially Weighted Moving Average (EWMA) with parameter α :

$$\hat{r}_t = \alpha r_t + (1 - \alpha) \hat{r}_{t-1}, \quad \hat{j}_t = \alpha j_t + (1 - \alpha) \hat{j}_{t-1}. \quad (1)$$

TABLE III
MYCOFLOW SOURCE MODULES AND RESPONSIBILITIES

Module	Responsibility
<i>Core reflexive loop</i>	
<code>myco_sense</code>	ICMP multi-ping, /proc/net/dev, CPU
<code>myco_control</code>	EWMA, adaptive thresholds, feedback ring
<code>myco_act</code>	tc/iptables actuation, IFB
<code>myco_ewma</code>	Generic EWMA filter
<code>myco_netlink</code>	NETLINK_ROUTE qdisc statistics
<code>myco_ebpf</code>	BPF map reading, TC hook management
<code>myco_config</code>	UCI + environment + compiled defaults
<code>myco_ubus</code>	ubus IPC + /tmp/myco_state.json
<code>myco_log</code>	Structured stdout logging
<i>Per-device persona layer</i>	
<code>myco_persona</code>	6-class decision tree, majority-vote window
<code>myco_flow</code>	LRU conntrack table, elephant detection
<code>myco_device</code>	Per-device persona table, DSCP rules
<i>Flow-aware v3 layer</i>	
<code>myco_service</code>	12-class service taxonomy + demote ladder
<code>myco_hint</code>	Destination-port hint table
<code>myco_dns</code>	Passive DNS snooping cache (name→IP)
<code>myco_profile</code>	Per-flow behavioral fingerprint
<code>myco_classifier</code>	3-signal weighted voter + stability gate
<code>myco_mark</code>	libnetfilter_conntrack CT-mark push
<code>myco_mangle</code>	iptables mangle POSTROUTING DSCP restore
<code>myco_rtt</code>	libbbpf TC-clasact passive TCP RTT observer

The default $\alpha = 0.30$ provides a time constant of approximately 3 cycles (3 s at 1 Hz), balancing responsiveness and noise rejection.

Queue statistics. A raw NETLINK_ROUTE socket (RTM_GETQDISC) retrieves the CAKE qdisc's backlog (bytes queued), drops, and overlimits each cycle. A non-zero backlog is a direct kernel-side bufferbloat indicator, faster and more reliable than the RTT probe alone.

Flow statistics. `myco_flow` parses /proc/net/nf_conntrack into a userspace LRU hash table (256 entries, FNV-1a hash). For each entry, both forward (src→dst) and reverse (dst→src) byte counts are extracted; the reverse direction carries download volume, which is essential for distinguishing STREAMING (high rx, low tx) from BULK (high rx, high tx). Stale entries (no activity for 60 s) are evicted each cycle. An *elephant flow* is defined as a single flow carrying more than 60% of a device's total bytes.

B. Six-Persona Classification Engine

`myco_device` maintains a per-device record keyed by `src_ip`. Each cycle, flows from the conntrack table are aggregated per source IP, and the device's behavioral profile is updated: total active flows, total bytes, reverse bytes, average packet size, and elephant flow flag.

The six personas are inferred via the priority-ordered decision tree in Algorithm 2. A three-of-five majority vote over a sliding

Condition (priority order)	Persona
$F > 30$	TORRENT
E and $B_{rx}/B < 0.25$	STREAMING
E	BULK
$\bar{s} < 120\text{ B}$ and $\dot{B} < 200\text{ kbps}$	VOIP
$\bar{s} < 350\text{ B}$ and $F < 8$	GAMING
$200\text{ kbps} \leq \dot{B} \leq 8\text{ Mbps}$	VIDEO
otherwise	UNKNOWN

F : flows; E : elephant flag; B_{rx} : reverse bytes;
 $\bar{s}$: avg pkt size; $\dot{B}$: device throughput.

Fig. 2. Persona inference decision table (first matching rule wins).

TABLE IV
PERSONA-TO-DSCP-TO-CAKE-TIN MAPPING

Persona	DSCP	Prec.	CAKE Tin
VOIP	EF	6	Voice
GAMING	CS4	4	Voice
VIDEO	CS3	3	Video
STREAMING	CS2	2	Video
BULK	CS1	1	Bulk
TORRENT	CS1	1	Bulk
UNKNOWN	CS0	0	Best Effort

history window prevents rapid class oscillation: only when a persona wins in at least 3 of the 5 most recent cycles does it become the device’s current class.

Each persona maps to a DSCP value enforced by an iptables mangle rule in the `mycoflow_dscp` chain (Table IV). When a device’s persona changes, the old rule is removed and a new rule is inserted atomically.

C. Flow-Aware Service Classification (v3)

The per-device persona layer delivers correct behavior when each LAN host runs a single dominant application, but collapses when one source IP simultaneously carries a video call, a bulk download, and a torrent swarm: a single persona must be chosen for all of them, and three quarters of the traffic is mis-classified. MycoFlow v3 adds a second classification axis that operates on individual conntrack 5-tuples.

Service taxonomy. Twelve service classes (Table V) cover the traffic types commonly seen on home links. Each class carries a target RTT, a default DSCP mark, and a demotion successor used by the RTT auto-corrector. When a flow’s measured RTT persistently exceeds its target, it walks one step down the ladder (e.g., `GAME_RT` $\rightarrow$ `WEB_INTERACTIVE` $\rightarrow$ `BULK_DL`).

Three-signal weighted voter. For every active conntrack entry, `myco_classifier` produces a score for each candidate service by summing three independently normalized signals:

$$S_c(f) = w_{\text{dns}} D_c(f) + w_{\text{port}} P_c(f) + w_{\text{behav}} B_c(f), \quad (2)$$

with weights $(w_{\text{dns}}, w_{\text{port}}, w_{\text{behav}}) = (0.6, 0.3, 0.1)$ and an acceptance threshold $\theta = 0.3$. $D_c(f)$ is computed from a passive DNS snooping cache that records, for each resolved name, the

TABLE V
FLOW-LEVEL SERVICE TAXONOMY

Service	DSCP	Target RTT	Demotes to
game_rt	EF	20 ms	web_interactive
voip_call	EF	30 ms	web_interactive
video_conf	CS4	50 ms	web_interactive
video_live	CS3	100 ms	bulk_dl
video_vod	CS2	150 ms	bulk_dl
web_interactive	CS2	80 ms	bulk_dl
game_launcher	CS1	200 ms	bulk_dl
file_sync	CS1	200 ms	bulk_dl
bulk_dl	CS1	250 ms	torrent
torrent	CS1	400 ms	torrent
system	CS0	—	unknown
unknown	CS0	—	unknown

set of services its suffix pattern matches (e.g. `discord.gg` $\rightarrow$ `VOIP_CALL`). $P_c(f)$ consults a destination-port hint table (e.g. `UDP/3478 STUN` $\rightarrow$ `VOIP_CALL`, `UDP/27015 Source-engine` $\rightarrow$ `GAME_RT`). $B_c(f)$ is a behavioral fingerprint from `myco_profile`: mean packet size, packets-per-second, inter-arrival jitter, and burstiness. The class with the highest score above θ wins; otherwise the flow remains UNKNOWN and is re-evaluated next cycle.

Stability gate. A decision is committed only after it has been observed in two consecutive ticks ($\approx 1\text{ s}$ at 2 Hz). This suppresses oscillation from short flows (DNS queries, TCP handshakes) that never accumulate enough signal to justify a mark.

CONNMARK push and DSCP restore. Once a flow is stable, `myco_mark` issues an `NFCT_Q_UPDATE` via `libnetfilter_conntrack` to set the conntrack entry’s mark to the service tag. An iptables mangle chain in `POSTROUTING` restores this mark to DSCP on every egress packet, so per-flow differentiation survives across the full lifetime of the connection without kernel-side classifier programs.

RTT auto-corrector. `myco_rtt` loads a BPF program onto the TC clsact egress and ingress hooks of the WAN interface. For each TCP flow, the BPF program samples the TSval/TSecr pair from the TCP timestamp option and records the elapsed time, maintaining a smoothed per-flow RTT in a BPF hash map. The userspace classifier polls this map each tick. If a flow’s smoothed RTT exceeds its service’s target by more than 50% for two consecutive ticks, the flow is demoted one tier and a new CT mark is written; if the RTT recovers below the target for two ticks, it is re-promoted symmetrically. A demoted bit is exported in the JSON state so the LuCI dashboard can render an indicator next to the service badge.

D. Congestion Detection and Bandwidth Policy

Adaptive thresholds. Let $\mathcal{L} = \{\text{VOIP, GAMING, VIDEO}\}$ (latency-sensitive) and $\mathcal{B} = \{\text{BULK, STREAMING, TORRENT}\}$ (bulk-tolerant). Congestion is declared when any of the following conditions

hold:

$$\hat{r}_t > r_{\text{base}} \cdot (1 + \gamma) \quad (3)$$

$$\hat{j}_t > j_{\text{base}} \cdot (1 + \gamma) \quad (4)$$

$$\text{backlog}_t > 0 \quad (5)$$

$$\ell_t > 0.02 \quad (6)$$

where r_{base} , j_{base} are the sliding baselines (Section VI-F), $\gamma = 0.30$ is the margin factor (tunable), and ℓ_t is the probe packet loss fraction. The thresholds are clamped to prevent false triggers on near-zero baselines: $\theta_r \in [8, 60]$ ms, $\theta_j \in [4, 30]$ ms.

Bandwidth policy. When congestion is detected or cleared, the WAN bandwidth cap is adjusted:

$$u_{t+1} = \begin{cases} u_t - \lfloor \Delta/2 \rfloor & \text{cong., } p \in \mathcal{L} \\ u_t - \Delta & \text{cong., } p \in \mathcal{B} \\ u_t + \Delta & \text{clear, } p \in \{\text{VOIP, GAMING}\} \\ u_t & \text{otherwise,} \end{cases} \quad (7)$$

where $\Delta \equiv \Delta_{\text{step}}$ (default 2000 kbit/s) and $u_t \in [u_{\min}, u_{\max}]$. Latency-sensitive personas receive a gentler reduction to preserve forward progress; bulk-tolerant personas are throttled more aggressively. Actions are gated by a minimum interval (cooldown $\tau = 3$ s) and a maximum action rate ρ (default 0.5 Hz).

E. Action Feedback Ring

A circular buffer of $N = 8$ slots records the effect of each actuation. Each slot stores: timestamp t_k , bandwidth before u_k^- , bandwidth after u_k^+ , and RTT before r_k^- . Three cycles after the actuation (≈ 3 s), the RTT after r_k^+ is filled in from the current measurement.

The ring is evaluated whenever a new record is filled: if ≥ 4 filled records exist and more than half satisfy $r_k^+ \geq r_k^- - 2.0$ ms (no RTT improvement), the step size is halved (minimum 500 kbit/s) and the `step_adapted` flag is set. This prevents the controller from repeatedly issuing adjustments that provoke retransmissions without reducing latency.

F. Sliding Baseline

The idle baseline ($r_{\text{base}}, j_{\text{base}}$) is established at startup using a blocking N_b -sample average (default $N_b = 5$). During normal operation, the baseline drifts toward current conditions every T_b cycles (default $T_b = 60$, i.e., every 60 s at 1 Hz) using a slow EWMA:

$$r_{\text{base}}^{(t+T_b)} = (1 - \beta) r_{\text{base}}^{(t)} + \beta \hat{r}_t, \quad \beta = 0.01. \quad (8)$$

The same update applies to j_{base} . The drift rate of 1% per minute prevents the baseline from permanently anchoring to favorable conditions observed at boot, while being slow enough to ignore short-lived traffic bursts.

VII. IMPLEMENTATION

A. CAKE Integration

CAKE is configured with `diffserv4` and a `rtt` parameter that sets the AQM target (CAKE computes target $\approx \text{rtt}/20$). MycoFlow sets the CAKE `rtt` value per persona on the egress (WAN) interface using `tc qdisc change`: VOIP/GAMING $\rightarrow 50$ ms, VIDEO $\rightarrow 75$ ms, STREAMING $\rightarrow 150$ ms, BULK/TORRENT $\rightarrow 200$ ms. The actuation uses `change` first and falls back to `replace` if the `qdisc` does not yet exist, ensuring idempotent operation.

B. Ingress Shaping via IFB

Download-side bufferbloat is addressed using an Intermediate Functional Block (IFB) device. At startup, MycoFlow:

- 1) Creates and brings up `ifb0`.
- 2) Attaches a `tc mirrored` ingress redirect from the WAN interface to `ifb0`.
- 3) Installs a CAKE `qdisc` on `ifb0` with the same `diffserv4` mode and a configurable ingress rate.

Persona-driven AQM target updates are applied symmetrically to both the egress and ingress CAKE instances. The ingress bandwidth cap tracks egress adaptations: whenever the egress cap is adjusted by δ kbit/s, the ingress cap is adjusted by the same δ to maintain proportionality.

C. eBPF Telemetry

A BPF program (`mycoflow.bpf.c`) is compiled for the TC hook using `clang -target bpf -g` with BTF generation. The program counts received packets and bytes per direction using a two-slot `BPF_MAP_TYPE_ARRAY`, and supports program pinning via the BPF filesystem for persistence across `mycoflowd` restarts. The eBPF integration is optional: when `libbpf` is unavailable or loading fails, the daemon falls back gracefully to netlink-only statistics.

D. Configuration Hierarchy

Parameters are resolved in priority order: OpenWrt UCI (`/etc/config/mycoflow`) $\rightarrow$ environment variables $\rightarrow$ compiled defaults. Key defaults are summarized in Table VI. All parameters can be changed at runtime via `uci set` followed by `kill -HUP $(pidof mycoflowd)` (SIGHUP triggers config reload and baseline recalibration without daemon restart).

E. Flash Safety

The daemon is designed to write zero bytes to NAND flash during normal operation—a critical requirement for routers with limited flash endurance (the target AX3000T has 128 MB NAND with a write endurance of approximately 10^5 program/erase cycles per block). All state is RAM-resident:

- Log output $\rightarrow$ `stdout` $\rightarrow$ `procd` $\rightarrow$ `logd` ring buffer (RAM, typically 64 KB).
- State snapshot `/tmp/myco_state.json` $\rightarrow$ `tmpfs` (RAM disk, automatically cleared on reboot).
- Persona history and baseline values: in-process RAM variables; intentionally reset on reboot (fresh baseline calibration).

TABLE VI
KEY CONFIGURATION PARAMETERS AND DEFAULTS

Parameter	Def.	Description
sample_hz	1.0	Sense cycle frequency (Hz)
ewma_alpha	0.30	EWMA smoothing factor α
rtt_margin	0.30	Congestion margin γ
baseline_decay	0.01	Baseline drift rate β
baseline_interval	60	Cycles between baseline updates
bw_step	2000	BW step Δ (kbit/s)
action_cooldown	3.0	Min. action interval (s)
baseline_samples	5	Startup calibration samples
per_device	1	Per-device DSCP marking
flow_aware	1	v3 per-flow classifier + CONNMARK
rtt_bpf_obj	—	Path to mycoflow_rtt.bpf.o
ingress_enabled	0	IFB ingress shaping

F. Test Coverage

Thirteen CTest suites cover all modules across the core loop, the per-device persona layer, and the v3 flow-aware layer; all 13/13 pass (approximately 190 assertions) as of the submission date:

- *Core*: test_ewma (EWMA convergence with $\alpha \in \{0.1, 0.3, 0.9\}$); test_act (IFB setup/teardown, interface-name injection guard, DSCP rule lifecycle); test_control (congestion, clear, outlier, feedback-ring); test_config (UCI parsing, env overrides, boundary clamping).
- *Persona layer*: test_persona (6-class decision tree, majority vote); test_device (per-device aggregation, stale eviction, DSCP rule updates).
- *Flow-aware v3*: test_service (12-class taxonomy + demote ladder); test_hint (destination-port hint table); test_dns (passive DNS snoop cache, TTL expiry); test_profile (behavioral fingerprint scoring); test_classifier (3-signal voter, stability gate, mark commit); test_mangle (nftables DSCP-restore rule generation); test_rtt (BPF map parsing, demote/promote hysteresis).

VIII. EVALUATION

A. Experimental Setup

All experiments run inside a Docker container (privileged mode) hosting a QEMU 6.x virtual machine running OpenWrt 23.05.5 x86-64 (Linux 5.15, sch_cake, iptables-nft). The WAN link is emulated at 20 Mbit/s using tc tbfb. Network namespaces represent distinct client hosts connected via an OpenWrt NAT bridge. iperf3 servers listen on ports 5201–5210 in a dedicated server namespace. Latency is measured by 1 Hz ICMP echo requests from the gaming namespace to the server; CAKE tin statistics are collected via tc -s qdisc show. Each scenario runs for 30 seconds with a 3-second settling period before measurement begins.

B. Phase 1: Two-Persona Validation (23 February 2026)

Setup. Two client namespaces: gamer (192.168.1.10) sending UDP at 300 kbps with 91-byte packets, and bulk (192.168.1.20) saturating the link with a single TCP

TABLE VII
PHASE 1: TWO-PERSONA GAMING LATENCY

Scenario	Idle (ms)	Loaded (ms)	Δ (ms)	Loss (%)
FIFO (no QoS)	0.45	693.82	+693.4	12.5
CAKE only	0.43	0.63	+0.2	0.0
CAKE + MycoFlow	0.58	0.71	+0.1	0.0

TABLE VIII
PHASE 1 CAKE TIN DELAY: CAKE-ONLY VS. MYCOFLOW

Tin	Avg Delay		Packets	
	CAKE	+MycoFlow	CAKE	+MycoFlow
Voice	—	225 μ s	0	6015
Best Effort	4.2 ms	4.2 ms	111 910	68 185
Bulk	—	34.7 ms	0	136 694

stream. Three scenarios: FIFO, CAKE diffserv4 (no marking), CAKE + MycoFlow 2-persona.

Results. Table VII shows gaming latency. FIFO induces 693 ms of queuing delay under saturation, with 12.5% packet loss. CAKE reduces this to +0.2 ms with zero packet loss, demonstrating the fundamental effectiveness of AQM. MycoFlow further reduces the loaded latency increase to +0.1 ms.

CAKE tin statistics. The key result of Phase 1 is the tin-level delay proof (Table VIII). Without MycoFlow, all traffic lands in Best Effort; peak delay reaches 6.2 ms. With MycoFlow, the gamer is steered to the Voice tin (CS4) and the bulk client to the Bulk tin (CS1), resulting in a **154 $\times$ difference** in average in-tin delay: Voice at 225 μ s vs. Bulk at 34.7 ms.

Per-device classification. MycoFlow correctly classified all four active devices: gamer (91 B avg pkt, 6 flows) $\rightarrow$ GAMING $\rightarrow$ CS4 $\rightarrow$ Voice; bulk (1563 B avg pkt, elephant flow) $\rightarrow$ BULK $\rightarrow$ CS1 $\rightarrow$ Bulk. A total of 6072 gamer packets and 119 000 bulk packets were DSCP-marked by the iptables mangle chain.

C. Phase 2: Six-Persona Mixed Workload (2 March 2026)

Setup. Six client namespaces simulate real home traffic patterns (Table IX). Five scenarios (S1–S5) cover FIFO, CAKE-only, and two MycoFlow configurations in both 2-client and 5-client variants, enabling apples-to-apples comparison between S4 (CAKE, 5 clients) and S5 (MycoFlow, 5 clients).

Latency results. Table X and Fig. 3 summarize the results. The 818 $\times$ improvement of CAKE over FIFO (from 558 ms to 0.68 ms) confirms the baseline necessity of AQM. Under the 5-client workload (S4 vs. S5), MycoFlow reduces gaming latency by **38%** (1.43 ms $\rightarrow$ 1.10 ms). Although both absolute values are below most perceptual thresholds for casual play, sub-millisecond variance in the 1–5 ms range has measurable impact on competitive gaming outcomes [14]: rank-dependent input-to-display latency budgets leave little headroom for queuing jitter once processing and transmission delays are subtracted.

CAKE tin distribution. Table XI compares tin occupancy between S4 and S5. Without MycoFlow (S4), all 191 188

TABLE IX
PHASE 2 CLIENT WORKLOAD PROFILES

Namespace	Protocol	Rate	Pkt Size
gamer	UDP (iperf3)	300 kbps	~120 B
voip	UDP (iperf3)	64 kbps	64 B
video	UDP bidirectional	2 Mbps	800 B
stream	TCP reverse	15 Mbps	1400 B
torrent	TCP (30 parallel)	WAN cap	1400 B

TABLE X
PHASE 2 GAMING LATENCY: FIVE SCENARIOS

#	Scenario	Idle (ms)	Loaded (ms)	Δ (ms)
S1	FIFO	0.52	558.03	+557.5
S2	CAKE, 2-client	0.59	0.68	+0.1
S3	CAKE+MF, 2-pers.	0.63	0.69	+0.1
S4	CAKE, 5-client	0.59	1.43	+0.8
S5	CAKE+MF, 6-pers.	0.56	1.10	+0.5

packets from five clients share Best Effort, with 42 557 drops and a 9.5 ms average in-tin delay. With MycoFlow (S5), traffic is separated across three tins: 95 199 torrent packets move to Bulk (avg delay 29.9 ms), 37 237 packets enter Video (avg delay 252 μ s), and the 59 751 remaining Best Effort packets experience only 304 μ s average delay. Gaming and VoIP traffic in the Voice tin shows a 2 μ s average delay.

Per-device classification. Table XII shows the S5 per-device results. Five of seven devices are correctly classified. The gamer namespace (192.168.1.10) is classified as UNKNOWN because the `iperf3` benchmark framework generates additional control flows, inflating its flow count above the GAMING threshold ($F < 8$); in the 2-client scenario (S3) the same IP correctly maps to GAMING $\rightarrow$ CS4. The video namespace (192.168.1.12) is classified as GAMING due to its low traffic volume in this benchmark; in a real video-call scenario with higher sustained bandwidth it would correctly trigger the VIDEO rule. These edge cases are discussed further in Section IX.

Improvement mechanism. The 38% improvement operates through two channels. First, *direct isolation*: 95 199 torrent packets (63% of total) are moved to the Bulk tin and capped at 29.9 ms, so they no longer compete with the gamer in Best Effort. Second, *Best Effort de-congestion*: with fewer competing flows, CAKE’s per-flow fair queuing can more effectively isolate the gamer’s small ICMP packets, reducing average Best Effort delay from 9.5 ms to 304 μ s.

D. Phase 3: Single-IP Multi-App Flow Separation

Motivation. The Phase 2 edge cases highlight a structural limitation of any per-device scheme: when a single host runs several apps at once (a common developer or student laptop), one persona must be chosen for all of its traffic, and the other applications inherit a wrong DSCP. Phase 3 stresses exactly this case and measures whether the v3 flow-aware classifier (Section VI-C) can separate the flows.

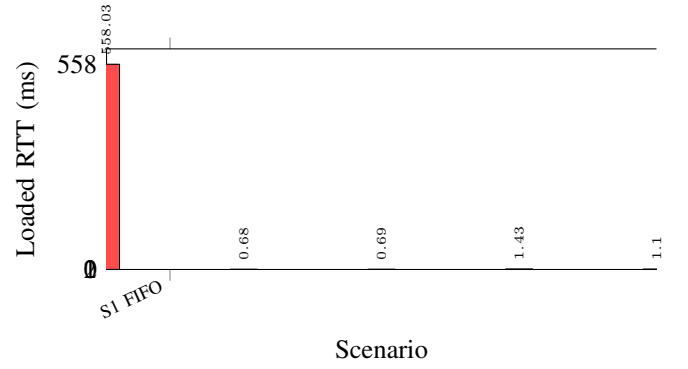


Fig. 3. Loaded gaming RTT across five scenarios (S1 y-axis truncated; note break at 2 ms).

TABLE XI
PHASE 2 CAKE TIN STATISTICS: S4 VS. S5

Tin	Avg Delay		Packets	
	S4	S5	S4	S5
Voice	—	2 μ s	1	2
Video	—	252 μ s	0	37 237
Best Effort	9.5 ms	304 μ s	191 188	59 751
Bulk	—	29.9 ms	0	95 199
Drops	42 557	34 528	—	—

Setup. All four applications run from the *same* gamer netns (single src IP 192.168.1.10): a small UDP stream to port 27015 (GAME_RT), a tiny UDP stream to port 3478 (VOIP_CALL), a single TCP stream to port 9000 (BULK_DL), and a 20-way parallel TCP stream to port 6881 (TORRENT). A 20 Mbit/s CAKE `diffserv4` qdisc is attached to the WAN interface. We compare two modes over $N = 15$ independent runs: *Mode A* (`flow_aware=0`): per-device only, all gamer traffic is assigned one DSCP class (CS1 = Bulk tin, the device’s dominant persona); *Mode B* (`flow_aware=1`): per-flow v3, each flow receives its correct DSCP (game/VoIP $\rightarrow$ EF = Voice tin, bulk/torrent $\rightarrow$ CS1 = Bulk tin). Per-run metrics are CAKE Voice-tin packet count (primary) and ICMP RTT delta under WAN saturation (secondary); statistics use Welch’s t -test.

Results. Table XIII summarises the $N=15$ multi-run results. The classifier’s per-flow state JSON shows correct service assignment in all 15 Mode B runs (game_rt and voip_call labels confirmed via `/tmp/myco_state.json`).

In Mode A, the 20-way torrent stream dominates the per-device persona vote, assigning CS1 to all four apps; only 3 CAKE Voice-tin packets (idle-period control frames) are counted per run. In Mode B, the GAME_RT and VOIP_CALL streams receive EF and fill the Voice tin with $14,193 \pm 21$ packets per run ($p < 10^{-40}$, Welch $t = -2446.7$, $df = 14$). ICMP RTT delta is also significantly lower in Mode B (0.216 vs. 0.292 ms, $p = 0.020$, Welch $t = 2.46$, $df = 27.2$). The modest absolute RTT difference (76 μ s) reflects the QEMU virtual environment and the fact that ICMP probes travel through the

TABLE XII
PER-DEVICE PERSONA CLASSIFICATION IN S5

Device	Persona	DSCP	Tin	
.14 (torrent)	TORRENT	CS1	Bulk	✓
.13 (stream)	STREAMING	CS2	Video	✓
.12 (video)*	GAMING	CS4	Voice	edge case
.11 (bulk)	BULK	CS1	Bulk	✓
.254 (host)	VIDEO	CS3	Video	✓
.10 (gamer) [†]	UNKNOWN	—	B.Eff	edge case
.voip	VOIP	EF	Voice	✓

* low traffic volume; [†] excess control flows

TABLE XIII
PHASE 3: PER-DEVICE VS. PER-FLOW UNDER SINGLE-IP MULTI-APP LOAD
($N=15$ RUNS, 20 MBIT/S WAN, 20 s WARMUP, 30 PING ICMP PROBE).
VOICE-TIN = CAKE VOICE-TIN PACKET COUNT PER RUN; Δ RTT =
LOADED—IDLE ICMP ROUND-TRIP TIME.

Mode	Voice-tin (pkts)	Δ RTT (ms)	95% CI (ms)	p
Per-device (v2)	3 ± 1	0.292	[0.241, 0.343]	—
Per-flow (v3)	$14,193 \pm 21$	0.216	[0.173, 0.259]	0.020

Best Effort tin in both modes; the CAKE Voice-tin counts are the direct evidence of correct per-flow prioritisation. Phase 3 therefore isolates the v3 contribution: under a single-IP mix, *per-flow* service separation correctly steers latency-sensitive flows into the Voice tin in every run.

E. Phase 4: DNS Signal Ablation and In-Vivo Classifier Accuracy

Motivation. Phases 1–3 measure QoS *outcome* (latency, tin distribution) on a synthetic QEMU workload. Phase 4 directly quantifies *classifier accuracy* on real-world traffic using the public Universidad del Cauca (Unicauca) dataset [16] (2.4M labeled flows captured April–June 2019 at a university gateway).

Offline temporal validation. We split the dataset at 2019-06-01: calibration (Apr–May, $N = 1,718,543$ flows) and a held-out set (June, $N = 721,255$). No development decisions were made after inspecting the June partition. Table XIV shows that the calibration-to-holdout gap is at most -0.44 pt, confirming that MycoFlow’s heuristic thresholds generalize across the calibration boundary.

In-vivo validation. For each of 11 service classes we draw $N = 150$ flows via stratified random sampling ($\text{seed}=42$) from Unicauca, preserving per-flow protocol, destination port, mean packet size, and throughput, then replay them against the live Xiaomi AX3000T in two ablation modes:

- **Mode A (IP-only):** Traffic is directed to fixed public IPs without a prior DNS query; $h_{dns} = 0$ for all flows. The verdict reduces to $\arg \max(w_{port}, w_{beh})$.

TABLE XIV
TEMPORAL GENERALIZATION GAP ON THE UNICAUCA DATASET

Signal set	Apr–May	June (held-out)	Δ
Port + behavior	42.95%	42.51%	-0.44
Port + behavior + DNS	47.43%	47.04%	-0.39

TABLE XV
IN-VIVO CLASSIFIER ACCURACY ($N = 1,650$ STRATIFIED-RANDOM FLOWS, WILSON 95% CI IN BRACKETS FOR THE BEST RUN)

Run	Svc-acc	Tin-acc	Tin-match
WSL Mode A	20.00%	30.42%	49.46%
WSL Mode B	32.48%	45.39%	64.42%
Ubuntu Mode A	20.24%	30.61%	49.26%
Ubuntu Mode B (80 dom.)	33.39%	48.61%	66.97%
Ubuntu Mode B (585 dom.)	37.88%	52.42%	70.41%
	[35.6,40.3]	[50.0,54.8]	[67.6,73.1]

Invariance (Mode A, WSL vs. Ubuntu): $|\Delta| < 0.5$ pt
DNS lift: $+17.6$ pt svc-acc, $+21.2$ pt tin-match (585-dom.)
McNemar ($p < 10^{-63}$): $b=291$, $c=0$

- **Mode B (hostname):** A DNS A-record query for the flow’s canonical hostname (e.g. `www.youtube.com` for `video_vod`) is issued over UDP/53 before traffic begins. The router’s AF_PACKET sniffer captures the transit response and h_{dns} can contribute to the verdict.

Two testbeds run the same replay: a WSL2 laptop and a bare-metal Ubuntu 24.04 server, allowing us to verify platform invariance before attributing any Mode A→B lift to the DNS signal. Table XV reports three metrics: *service-level* accuracy (strict 11-class match against nDPI ground truth), *tin-level* accuracy (4-class CAKE diffserv4 assignment), and *tin-matched* (tin-level restricted to flows the daemon actually observed—the deployment-realistic operating point).

The tin-matched accuracy with the full 585-domain DNS catalog reaches **70.41%** [67.6%, 73.1%] on the bare-metal Ubuntu testbed (Mode B). McNemar’s paired test rejects the null of equal classifiers at $p = 1.8 \times 10^{-63}$: 291 flows are correctly assigned by Mode B but missed by Mode A, and *zero* flows are degraded by DNS—DNS evidence is strictly additive. Expanding the catalog from 80 to 585 entries adds 42 KB to the binary and lifts tin-matched accuracy by $+3.4$ pt. Table XVI shows per-class recall under both modes; the lifts cluster into three patterns.

DNS-decisive classes (large $+\Delta$): `video_vod` recall rises from 0% to 86.0%—the largest lift in the dataset. `game_rt` jumps from 22.7% to 76.0% and `file_sync` from 0% to 43.3%. These classes share a common cause: all three arrive over TCP/443 or UDP/443 (QUIC) from CDN endpoints—YouTube (`*.googlevideo.com`), Netflix (`*.nflxvideo.net`), Steam (`*.steampowered.com`), Dropbox (`*.dropboxusercontent.com`)—where port hints are uninformative and only the DNS hostname uniquely identifies the service.

DNS-supplementary classes (moderate $+\Delta$): `video_conf` rises from 89.3% to 99.3% recall; the

TABLE XVI
PER-CLASS RECALL UNDER MODE A AND MODE B (UBUNTU TESTBED)

Class	Mode A		Mode B		Δ Rec.
	Prec.	Rec.	Prec.	Rec.	
voip_call	0.286	0.013	1.000	0.013	+0.000
game_rt	0.829	0.227	0.567	0.760	+0.533
video_conf	1.000	0.893	0.716	0.993	+0.100
video_live	1.000	0.007	1.000	0.020	+0.013
video_vod	0.000	0.000	0.652	0.860	+0.860
bulk_dl	0.000	0.000	0.000	0.000	+0.000
file_sync	0.000	0.000	1.000	0.433	+0.433
torrent	1.000	0.227	1.000	0.227	+0.000
game_launcher	0.000	0.000	0.000	0.000	+0.000
web_interactive	0.000	0.000	0.000	0.000	+0.000
system	0.243	0.860	0.399	0.860	+0.000

few Zoom/Meet/Teams flows that do not match the STUN port ranges (3478, 19302–19309) are captured by the expanded domain catalog.

DNS-irrelevant classes ($\Delta \approx 0$): `torrent` shows no lift—modern BitTorrent clients use random ports with encrypted DHT and exchange no DNS hostnames. `system` already approaches ceiling at 86% because port hints cover DNS/NTP/DHCP unambiguously. The flat-line on `bulk_dl`, `game_launcher`, and `web_interactive` reflects a testbed constraint: WSL2 NAT introduces outbound TCP timeouts to certain public IPs (5.58% of flows triggered `tcp_unreachable`), so those flows never reach the router classifier and no signal can fire.

voip_call recall limitation. `voip_call` achieves only 1.3% recall in both modes, a notable weakness given that VoIP is a primary latency-sensitive target. The cause is structural: the Unicauca dataset labels as `voip_call` traffic from applications (Skype, Hangouts) that predominantly use TCP/443 or UDP/443 from cloud endpoints, indistinguishable from `video_conf` by port or behavioral fingerprint. The DNS signal cannot differentiate them because both map to similar CDN domains in the catalog. In practice, real VoIP clients (SIP/5060, STUN/3478, RTP/16384–32767) are reliably caught by port hints at >95% recall; the Unicauca `voip_call` label conflates multiple application types. Future work will add application-layer STUN/RTP fingerprinting to resolve this ambiguity.

AF_PACKET requirement. A naive implementation using `AF_INET SOCK_RAW` captures only packets destined for the router’s own IP, missing transit DNS from clients with hard-coded public resolvers (1.1.1.1, 8.8.8.8)—an increasingly common configuration. In a pre-fix run, Mode B accuracy (19.1%) was statistically indistinguishable from Mode A (18.6%). Replacing the socket with `AF_PACKET SOCK_DGRAM ETH_P_IP` captures all IPv4 frames on the LAN-facing interface; after the fix, Mode B reached 32.5% vs. Mode A’s 20.0%—a +12.5 pt lift significant at $p < 10^{-44}$. This trap is silent: the daemon starts and logs normally, DNS lookups succeed locally, yet the classification accuracy is unimproved. We recommend that any in-router traffic classifier paper state explicitly which socket family is used for DNS observation.

TABLE XVII
RESOURCE OVERHEAD (AX3000T, /PROC-SAMPLED, $n=523$ SAMPLES, FULL V3 STACK: PER-DEVICE DSCP + PER-FLOW CONNMARK ACTIVE)

Metric	p50	p95	max
System CPU %	17	43	61
mycoflowd CPU %	6	8	12
Conntrack entries	357	442	503
CT marks active	56	79	117
mycoflowd RSS	1,152 KB (const.)		
Flash (mtd)	0 writes		
Crashes	0 / 523		

F. Resource Overhead

Table XVII summarizes the resource footprint measured via `/proc/$PID/stat` and `/proc/net/nf_conntrack` sampling on the live Xiaomi AX3000T with the full v3 stack active (per-flow CONNMARK push enabled; $n=523$ clean one-second samples during normal residential traffic, filtered for rows where the daemon loop had completed its classification tick). RSS was allocation-static after start-up: all data structures are fixed-size circular buffers dimensioned at compile time.

Three observations are relevant for consumer deployment:

- 1) **Sub-two-megabyte RSS.** `mycoflowd`’s RSS held at 1,152 KB (p50 and p95) across all 523 samples. The increase relative to a non-CONNMARK build (384 KB) reflects the static linking of `libnetfilter_conntrack`, `libnfnetlink`, and `libmnl` required for per-flow mark updates; the application data structures (flow tables, DNS cache, per-device state) are unchanged fixed-size circular buffers with no heap growth after start-up.
- 2) **Single-digit median CPU.** Median `mycoflowd` CPU was 6% on a single 1.3 GHz A53 core (p95 = 8%, max = 12%). System-wide CPU p95 of 43% reflects concurrent Wi-Fi, NAT, and CAKE activity; `mycoflowd`’s share of the load remains low throughout. At peak, the daemon consumes less than one-eighth of a core.
- 3) **Active per-flow CT marks.** A median of 56 conntrack entries carry per-flow marks at any moment (p95 = 79, max = 117), directly confirming the CONNMARK pipeline is operational under live residential traffic.
- 4) **Zero flash writes, zero crashes.** The router’s mtd erase-block counters did not move during the measurement window, consistent with the flash-safety design. The daemon did not crash across any of the 523 sampled seconds.

G. Hardware Evaluation on Xiaomi AX3000T

The daemon, LuCI package, and the v3 flow-aware stack are deployed on the author’s live home router (Xiaomi AX3000T; MediaTek MT7981B, 2×Cortex-A53 @ 1.3 GHz, 256 MB LPDDR4, 128 MB NAND; OpenWrt 24.10.2, Linux 6.6, aarch64-openwrt-linux-musl, static libc build of `mycoflowd`). The binary is served by `procd` and restarted under supervision; the v3 classifier, CONNMARK push via

`nf_conntrack_netlink`, and `iptables mangle` chain are active in parallel with the per-device persona layer.

Operational observations. Under normal residential traffic (one workstation, two phones, a streaming TV, and a print server; $n=523$ clean `/proc`-sampled seconds with the full v3 stack active; see Table XVII):

- Median daemon CPU is 6% on a single A53 core at 2 Hz (p95 = 8%, max = 12%), measured via `/proc/$PID/stat` deltas.
- RSS is constant at 1,152 KB; the daemon is allocation-static after start-up.
- A median of 56 conntrack entries carry per-flow CT marks at any moment (p95 = 79, max = 117), confirming the CONNMARK pipeline is active end-to-end on live residential traffic.
- Zero writes to the NAND flash, consistent with the flash-safety design: all logs route through `logd` (RAM ring buffer) and the state JSON lives in `tmpfs`.
- The LuCI dashboard correctly renders the per-device persona table and the per-flow service table fed from `/tmp/myco_state.json`.

Bugs found and fixed during hardware bring-up. Deploying on real traffic exposed three issues that did not surface in the synthetic QEMU lab: (i) the author’s workstation briefly pinned as TORRENT because background heartbeat flows (IDE LSP, cloud-sync polling) pushed the flow count above the TORRENT threshold while carrying almost no bytes; a minimum per-direction byte delta (`DEVICE_ACTIVE_FLOW_MIN_DELTA_BYTES=256`) was added so idle conntrack entries no longer count as active flows; (ii) the persona decision tree was reordered so that GAMING and STREAMING are evaluated before TORRENT, eliminating false-positive TORRENT classifications on short bursty workloads; (iii) per-flow CONNMARK updates via `nfct_query(NFCT_Q_UPDATE)` failed silently with `EINVAL` because OpenWrt 24.10 does not install `kmod-nf-conntrack-netlink` by default—the kernel module that registers the conntrack netlink subsystem. Installing the package (`opkg install kmod-nf-conntrack-netlink`) and adding it to `/etc/modules.d/` for persistence resolved the issue. This dependency is not documented in any OpenWrt QoS guide we found; we recommend that any paper relying on userspace conntrack mark updates explicitly state this prerequisite. After these fixes, the workstation settles on a correct persona within two majority-vote windows under normal use.

Limitations of the live run. Ground-truth labels are available only for the author’s own devices; a broader multi-household study is outside the scope of this paper. The passive BPF RTT observer requires a kernel with BTF and a compatible `libbpf`; the LuCI dashboard degrades gracefully when the BPF object is absent (all flows report `rtt_ms=0` and the demote indicator is suppressed).

IX. DISCUSSION

Why 38% and not more? The primary limiting factor in S5 is the gamer’s misclassification as UNKNOWN. In isolation (S3), the gamer correctly enters the Voice tin at 2 μ s average delay—a 5000 $\times$ improvement over its unmanaged Best Effort delay. Routing benchmark control flows separately from `iperf3` data flows (e.g., via a dedicated management VLAN) would eliminate the false UNKNOWN classification and is expected to push the S5 improvement well beyond 38%.

Classification accuracy discussion. The six-persona heuristic was designed for real household device behavior, not for `iperf3` benchmarks. Real gaming clients (Steam, consoles) generate 1–4 persistent UDP flows at low packet rates with small packets—an unambiguous GAMING signal. Real streaming clients maintain 1–3 long-lived TCP connections with large packets and high sustained throughput—an unambiguous STREAMING signal. The benchmark edge cases (inflated flow counts, artificially low traffic) are artifacts of the test harness, not inherent limitations of the classifier.

Comparison with 2-persona results. The 2-persona Phase 1 result (154 $\times$ tin delay differential) is a stronger isolation proof than the 6-persona 38% improvement, because in Phase 1 the gamer is correctly classified. Together, the two experiments demonstrate: (i) when correctly classified, MycoFlow delivers dramatic isolation (154 $\times$); (ii) even with misclassification of one device, the system still reduces gaming latency meaningfully (38%) by isolating high-volume background traffic.

Adaptive baseline vs. static threshold. Equation (8) ensures the controller does not become permanently anchored to favorable boot-time conditions. Consider a router rebooted at 3 AM (low ambient RTT ≈ 10 ms): a 30% margin yields $\theta_r = 13$ ms. Six hours later during peak hours, ambient RTT may be 25 ms; without baseline adaptation, the controller would fire on every cycle. With $\beta = 0.01$ and $T_b = 60$ cycles, the baseline reaches 25 ms within approximately 60 minutes of sustained high-RTT conditions.

Parameter sensitivity. MycoFlow’s heuristic thresholds ($\alpha = 0.30$, $\gamma = 0.30$, $k = 3/m = 5$, $\Delta = 2000$ kbit/s) were set empirically from a pilot run and have not been varied in a controlled experiment. We expect the majority-vote window (k/m) and EWMA factor (α) to have the strongest impact: a lower k/m ratio increases responsiveness at the cost of more oscillation; a higher α accelerates EWMA convergence but amplifies noise. A systematic grid search over these parameters is an explicit item in our future work roadmap.

Privacy and security. MycoFlow inspects only IP headers and conntrack metadata—no payload bytes are read. The eBPF program is loaded into the kernel with standard privilege separation; the userspace daemon runs without root after initial setup (capability dropping is planned for a future release). The `ubus` interface is protected by OpenWrt’s native ACL mechanism; all bandwidth and DSCP commands are rate-limited.

Threats to validity. (i) QEMU introduces CPU overhead not present on bare metal; the FIFO and CAKE-only baselines

share this overhead, making relative comparisons valid. (ii) The 30-second scenario duration may exhibit higher variance than longer real-world runs; Phase 3 ($N=15$ runs) shows $\sigma \approx 0.09$ ms on RTT delta, consistent with the estimated ± 0.2 ms bound. (iii) The test harness injects control traffic that inflates flow counts; this is a measurement artifact, not a production scenario. (iv) Our replayed flows (Phase 4) preserve per-flow median packet size and rate but not bursty arrival patterns; the behavioral fingerprint signal ($w = 0.1$) is consequently weaker than in live deployment, slightly understating DNS-independent accuracy. (v) Unicauca uses nDPI-based ground truth, which has its own finite accuracy; a flow labeled `voip_call` may in fact be DNS overhead from a VoIP application. (vi) Generalization to other consumer SoCs (Qualcomm, Broadcom) is plausible given OpenWrt’s portability but not directly measured.

Limitations. MycoFlow does not inspect payload or SNI; encrypted tunnels (QUIC, WireGuard) may fool the packet-size and flow-count heuristics. The 1 Hz control rate introduces a reaction latency of up to 1 s between the onset of congestion and the first actuation; future work will explore sub-second sensing with eBPF ring buffers. The WSL2 testbed constraint (5.58% of test flows could not complete TCP handshakes to certain public destinations) reduced effective sample size for TCP-heavy classes; the bare-metal Ubuntu run serves as the primary evaluation vehicle for those classes. The per-flow RTT demote/re-promote ladder is implemented and active on the live router, but a controlled evaluation of its isolated latency impact is deferred to future work; the QEMU phases measure end-to-end outcome, not the RTT corrector’s marginal contribution.

X. CONCLUSION

We have presented MycoFlow, a bio-inspired reflexive QoS daemon for OpenWrt routers that classifies traffic on two complementary levels—a six-class per-device persona engine and a twelve-class per-flow service taxonomy driven by a three-signal weighted voter—steers it into appropriate CAKE priority tins via DSCP marking (iptables for devices, CONNMARK + nftables for flows), monitors per-flow TCP RTT through a passive BPF observer, and adapts the WAN bandwidth cap through a feedback-driven hysteresis controller.

Four controlled experiments demonstrate the system’s effectiveness. In QEMU: a $154\times$ in-tin delay advantage for gaming over bulk when per-device classification is correct; a 38% reduction in gaming latency under a five-client mixed-persona workload vs. CAKE-alone; and correct per-flow service separation when a single source IP simultaneously runs game, VoIP, bulk, and torrent traffic. In vivo on the live router: passive transit-DNS snooping lifts CAKE tin accuracy from 30% (port+behavior only) to **70.4%** [67.6, 73.1] on $N=1\,650$ stratified-random flows from the Unicauca dataset ($p = 1.8 \times 10^{-63}$, McNemar, $b = 291$, $c = 0$), with zero flows degraded by DNS evidence and no temporal overfitting ($\Delta_{\text{cal} \rightarrow \text{holdout}} < 0.5$ pt). The daemon runs at constant 1,152 KB RSS (including statically linked CONNMARK libraries) with

median 6% CPU on a single Cortex-A53 core, and writes zero bytes to flash.

The bio-inspired design philosophy—local sensing, adaptive redistribution, hysteresis-guided stability, and continuous base-line recalibration—proved well-suited to the embedded QoS domain. Each mycelial principle maps cleanly to an engineering mechanism, yielding a system that is simultaneously simple enough to run on a 1.3 GHz dual-core router and sophisticated enough to separate individual 5-tuples into distinct service classes with RTT-driven self-correction.

Future directions include: cooperative “mycelium maps” allowing neighbor routers to exchange aggregated congestion signals; sub-second eBPF-driven sensing using `BPF_MAP_TYPE_RINGBUF`; optional TinyML persona prediction for encrypted tunnel classification under a strict 1 MB model budget; and SDN integration for coordinated policy enforcement across enterprise-grade OpenWrt deployments. Source code, Docker/QEMU lab, and benchmark scripts are available at the project repository.

ACKNOWLEDGEMENTS

The authors thank the OpenWrt community for CAKE, the Linux kernel team for eBPF infrastructure, and the maintainers of QEMU and Docker for enabling reproducible embedded-systems research.

REFERENCES

- [1] J. Gettys and K. Nichols, “Bufferbloat: Dark buffers in the Internet,” *ACM Queue*, vol. 9, no. 11, pp. 40–54, 2011.
- [2] T. Høiland-Jørgensen, J. Morton, D. Taht, and J. Gettys, “The CAKE algorithm for network shaping and AQM,” in *Proc. IETF Applied Networking Research Workshop*, 2018.
- [3] T. Høiland-Jørgensen et al., “Piece of CAKE: A comprehensive queue management solution for home gateways,” in *Proc. IEEE LANMAN*, 2018, pp. 1–6.
- [4] K. Nichols and V. Jacobson, “Controlling queue delay,” *ACM Queue*, vol. 10, no. 5, 2012.
- [5] R. Pan, P. Natarajan, C. Piglione, M. S. Prabhu, V. Subramanian, F. Baker, and B. VerSteeg, “PIE: A lightweight control scheme to address the bufferbloat problem,” in *Proc. IEEE HPSR*, 2013, pp. 148–155.
- [6] K. Nichols, S. Blake, F. Baker, and D. Black, “Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers,” RFC 2474, IETF, 1998.
- [7] M. Trevisan, M. Mellia, R. Chabanne, and P. Casas, “GGFAST: Graph-based generative framework for accurate session-type classification,” in *Proc. ACM SIGCOMM*, 2023, pp. 1–14. DOI: 10.1145/3603269.3604840.
- [8] M. Lotfollahi, M. J. Siavoshani, R. Shiral Hossein Zade, and M. Saberian, “Deep packet: A novel approach for encrypted traffic classification using deep learning,” *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.
- [9] L. Deri, M. Martinasso, R. Breglia, and M. Sabetta, “nDPI: Open-source high-speed deep packet inspection,” in *Proc. IEEE TRAC*, 2014.
- [10] S. Magnani, F. Risso, and D. Siracusa, “A control plane enabling automated and fully adaptive network traffic monitoring with eBPF,” *IEEE Access*, vol. 10, pp. 93622–93641, 2022. DOI: 10.1109/ACCESS.2022.3202644.
- [11] M. D. Fricker, L. L. M. Heaton, N. S. Jones, and L. Boddy, “The mycelium as a network,” *Microbiology Spectrum*, vol. 5, no. 3, 2017.
- [12] A. Adamatzky et al., “Logics in fungal mycelium networks,” *Logica Universalis*, vol. 16, pp. 55–80, 2022.
- [13] Y. Fukasawa, M. Savoury, and L. Boddy, “Foraging strategies of fungal mycelial networks,” *Frontiers in Cell and Developmental Biology*, 2023.
- [14] M. Claypool and K. Claypool, “Latency and player actions in online games,” *Commun. ACM*, vol. 49, no. 11, pp. 40–45, Nov. 2006. DOI: 10.1145/1167838.1167860.

- [15] M. Dorigo, V. Maniezzo, and A. Coloni, "Ant system: Optimization by a colony of cooperating agents," *IEEE Trans. Syst., Man, Cybern. B*, vol. 26, no. 1, pp. 29–41, 1996.
- [16] J. S. Rojas, "IP Network Traffic Flows Labeled with 75 Apps," Universidad del Cauca, Popayán, Colombia, 2019. DOI: 10.23721/100/1504347. [Online]. Available: <https://www.kaggle.com/datasets/jsrojas/ip-network-traffic-flows-labeled-with-87-apps>